EchoFuzz: Empowering Smart Contract Fuzzing with Large Language Models

Juanen Li
Tsinghua University
Beijing, China
Beihang University
Beijing, China
Beijing Normal University
Zhuhai, China
juanen.li.se@gmail.com

Peng Qian
GoPlus Security & Zhejiang University
Hangzhou, China
messi.qp711@gmail.com

Guanyan Li
University of Oxford
Oxford, United Kingdom
guanyan.li.cs@gmail.com

Rui Wang
Beijing Normal University
Zhuhai, China
wangrui0921666@gmail.com

Peixin Wang
East China Normal University
Shanghai, China
pxwang@sei.ecnu.edu.cn

Zhiqing Tang
Beijing Normal University
Zhuhai, China
zhiqingtang@bnu.edu.cn

Fuchen Ma
Tsinghua University
Beijing, China
fuchenma525@gmail.com

Yuanliang Chen*
Tsinghua University
Beijing, China
sard.chen@gmail.com

Lun Zhang
GoPlus Security
Hangzhou, China
allen@gopluslabs.io

Abstract

Smart contracts, serving as the cornerstone of decentralized applications, autonomously manage trillion-dollar digital assets, making them attractive targets for attacks. Fuzzing has emerged as a promising technique for detecting vulnerabilities in smart contracts, yet existing methods face two main challenges. (1) The logical gap in state transitions and combinatorial redundancy hinders effective tradeoffs between bug detection efficiency and state space exploration cost, leading to critical execution paths to be overlooked. (2) Rule-based sequence mutation strategies suffer from path redundancy and inadequate guidance from contract logic, resulting in performance bottlenecks that stall the exploration of in-depth vulnerability-oriented paths.

To tackle these challenges, we propose EchoFuzz, an LLM-guided fuzzing framework introducing Vulnerable Function Call Sequences (VFCS) - minimal, behavior-preserving execution paths that expose bugs through key state transitions. EchoFuzz consists of two key procedures. First, we develop a chain-guided LLM approach, that combines static analysis with logical understanding to generate contract-specific VFCS candidates that eliminate combinatorial redundancy. Second, we adopt an iterative fuzzing strategy that uses LLMs with real-time feedback to adaptively steer fuzzer toward uncovered branches. Experiments show EchoFuzz outperforms state-of-the-art methods, achieving 29% higher branch coverage and detecting 62% more vulnerabilities. It also found 37 previously unknown vulnerabilities in real contracts, showing strong practicality.

Keywords

Smart Contract, Fuzzing, Large Language Model, Bug Detection

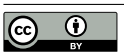
ACM Reference Format:
Juanen Li, Peng Qian, Guanyan Li, Rui Wang, Peixin Wang, Zhiqing Tang, Fuchen Ma, Yuanliang Chen, and Lun Zhang. 2026. EchoFuzz: Empowering Smart Contract Fuzzing with Large Language Models. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3773166

1 Introduction

Smart contracts are blockchain-based programs that autonomously execute predefined terms, serving as the cornerstone of decentralized applications across various domains [6, 25, 38]. Pioneering platforms like Ethereum host millions of contracts [30, 31, 50], revolutionizing domains like digital finance [2, 10], supply chain [3, 35], and crowdfunding [22, 54]. They now manage trillion-dollar assets, but vulnerabilities have caused over $10 billion [17, 36, 56] in losses, underscoring the critical need for robust security [32].

To ensure the security of smart contracts, a variety of vulnerability detection techniques are employed [1, 5]. Among these, fuzzing [27] is considered one of the most powerful and encouraging methods for automated vulnerability detection [28, 39]. Fuzzing detects vulnerabilities in smart contracts by repeatedly generating and executing a wide range of random or targeted inputs to test the contract's functions. During this process, the fuzzer is able to explore various execution paths within the contract to identify abnormal or unexpected behaviors [4].

* *Yuanliang Chen is the corresponding author.*



ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3773166

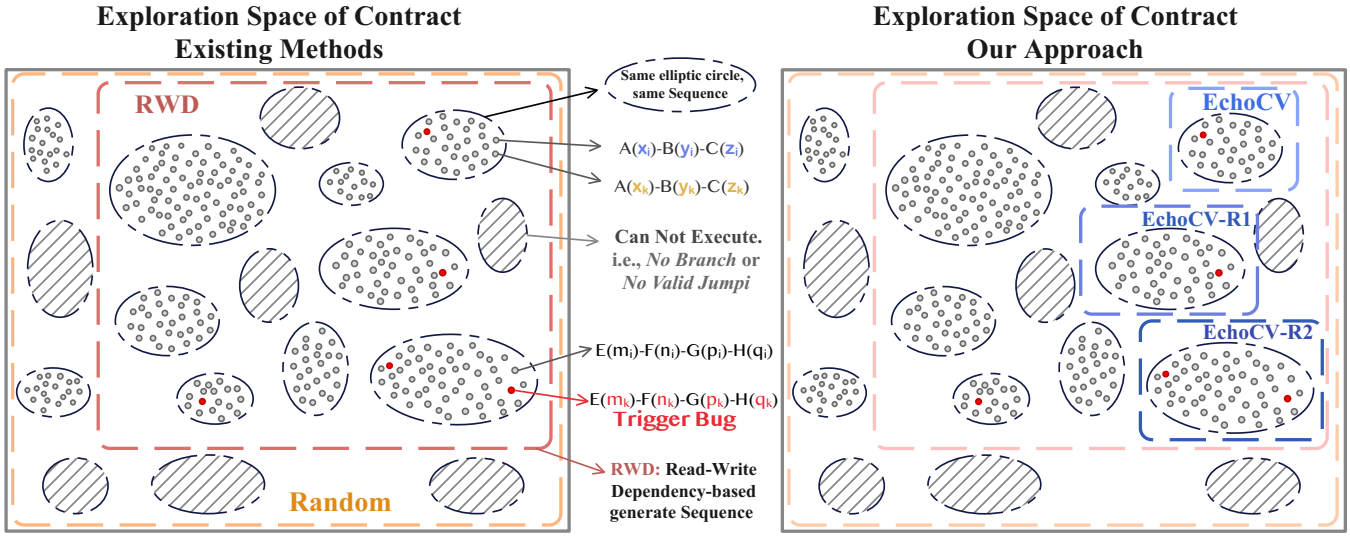


Figure 1: Each ellipse represents a specific sequence, and the points within the ellipses denote test cases with parameters that into fuzzing; gray points indicate test cases that execute normally, while red points represent test cases that reveal vulnerabilities. The gray-shaded areas within the ellipses signify sequences that are non-executable, labeled as *No Branch* or *No Valid Jumpi*.

However, due to its transaction-centric nature, fuzzing smart contracts presents unique challenges. Heuristically, the essence of smart contract fuzzing is to explore the possible execution sequences, i.e., $[f_1(a_1) \rightarrow f_2(a_2) \rightarrow \cdots \rightarrow f_n(a_n)]$, where $f_i()$ denotes a function call in the contract and a_i is its argument. The goal is to check if the final state reveals vulnerabilities with some given oracles. Despite advances in argument generation, creating quality call sequences remains hard due to vulnerability interdependencies on call orders. Poorly designed sequences may fail to expose complex vulnerabilities, as rigid patterns restrict access to essential execution paths. Simple sequences ignore contract logic, while full transaction call record simulation is computationally expensive, hindering the exploration of in-depth and hidden vulnerabilities. Thus, generating effective sequences faces two key challenges.

Challenge 1: Balancing Vulnerability Detection Probability and Exploration Space Cost. State space explosion arises from software engineering limitations: logical gaps in state transitions and combinatorial redundancy. Traditional methods track data dependencies but miss key functional relationships like state-offsetting interactions. This creates a trade-off between exploration cost and detection probability. Oversimplified state spaces miss critical paths, while over-extended ones lead to high computational cost without significant gains in vulnerability detection.

Challenge 2: Evolving State Space Exploration Beyond Fuzzing Bottlenecks. Fuzzing often plateaus, with fewer new vulnerabilities found over time. Conventional methods either continue within the same state space or use rule-based mutations, which often produce redundant sequences. These approaches revisit previously explored areas instead of probing new, underexplored regions. Without logic-driven adaptation, they lack targeted exploration, reducing efficiency and new vulnerability discovery.

Existing methods struggle to address these challenges effectively. As in Figure 1, random generation techniques (orange box) produce arbitrary outputs rarely finding meaningful patterns in complex contracts. Read-write dependency-based (RWD) methods (maroon box) analyze sequence dependencies via contract read/write operations. While RWD methods improve on random generation by filtering invalid sequences, their rule-based approaches cause over-generalization and imprecision with contract-specific logic. RWD methods, such as constraint-based symbolic execution [45], reduce the exploration space by enforcing constraints, but they are computationally expensive and struggle with non-deterministic behavior. Similarly, other RWD approaches, including dynamic taint [23] and evolutionary algorithms [26], overproduce scenarios but miss nuanced vulnerabilities. These methods often overlook in-depth bugs, converge poorly, and lack adaptability. Recent attempts using reinforcement learning [44] also struggles with defining robust reward functions and generalizing across diverse contract types.

Our Approach. To address these challenges, we introduce EchoFuzz, a fuzzing framework that leverages Large Language Models (LLMs) [55] for logical reasoning. Unlike traditional methods, it uses LLM-guided strategies to deeply explore smart contract state spaces. Central to our approach is the introduction of Vulnerable Function Call Sequences (VFCS), a novel abstraction that identifies minimal, logically coherent paths to directly trigger vulnerabilities, overcoming problems such as the logical gap and combinatorial redundancy. VFCS is defined by a series of dependent function calls that target areas most likely to contain vulnerabilities while minimizing unnecessary exploration. Inspired by expert analysis, we use LLMs to interpret code and predict vulnerable paths via state transitions. The iterative nature of our approach, guided by real-time feedback, ensures efficient vulnerability detection. EchoFuzz integrates two critical components that guide the fuzzing process through contract logic analysis and iterative refinement to efficiently and effectively identify vulnerabilities.

Procedure 1: Chain-Guided Candidate VFCS Generation. For *Challenge 1*, we generate candidate VFCSs (cf. Section 4.1 for details) via an LLM-driven chain-guided process. Using code comprehension, static analysis, and few-shot prompting, the LLM targets high-risk paths and key state transitions to reveal vulnerabilities. This context-aware method narrows the exploration space, ensuring critical VFCSs are found. Experiments show this method generates targeted sequences, improving exploration efficiency and vulnerability detection. As shown in Figure 1, EchoFuzz generates candidate VFCSs (denoted as **EchoCV**, the blue dashed box) through LLM-guided analysis, capturing areas with the highest likelihood of vulnerabilities.

Procedure 2: Feedback-Driven LLM-Guided Iterative Fuzzing. For *Challenge 2*, we use an iterative fuzzing strategy with real-time feedback and LLM-based code comprehension. It analyzes runtime feedback to find unexplored or poorly covered paths. Based on this feedback, the LLM analyzes the logic of branch patterns and generates focused sequences to drive the fuzzer towards the uncovered areas, preventing stagnation during exploration. As in Figure 1, this iterative approach generates new candidate sequences, EchoCV-N1 and EchoCV-N2, targeting previously uncovered branches, enhancing coverage and vulnerability detection.

We conduct experiments to evaluate the effectiveness of our proposed framework EchoFuzz across three datasets, including large-scale real-world contract blockchain projects. Empirical results strongly support our claim that EchoFuzz mitigates key challenges in smart contract fuzzing. Specifically, EchoFuzz achieves an average improvement in 29.19% in branch coverage and detects 62.77% more vulnerabilities compared to the current state-of-the-art. Encouragingly, EchoFuzz uncovers 37 previously unknown vulnerabilities in various contract projects, showcasing its strong performance and practical value in enhancing security.

Main Contributions. To summarize, the key contributions of this work are as follows:

- We present EchoFuzz, the first fuzzing framework to use chain-guided LLMs for smart contract logic, focusing on vulnerable function call sequences to pinpoint the paths that trigger vulnerabilities, overcoming the limitations of traditional methods.
- EchoFuzz combines contextual and static analysis to enhance VFCS generation and uses a real-time feedback-driven, iterative fuzzing process to dynamically refine strategies, expanding coverage and boosting vulnerability detection.
- Experimental results show that EchoFuzz outperforms current state-of-the-art fuzzers in code coverage and vulnerability detection. In the spirit of open science [1] , we release our framework for further research and development.

2 Background

Ethereum Smart Contract. Smart contracts, conceptualized by Nick Szabo in 1994 [9], are self-executing programs on blockchains like Ethereum that enforce predefined terms without intermediaries. These attributes enable their application in decentralized finance (DeFi) [48, 53], voting systems [34], and non-fungible tokens (NFTs) [7, 40, 46]. However, the substantial digital assets they hold attract sophisticated attacks, causing billion-dollar losses, which makes robust vulnerability detection crucial.

[1] https://github.com/iceray00/EchoFuzz

```solidity
contract Fundraiser {
    uint256 phase = 0;
    uint256 goal;
    uint256 invested;
    address owner;
    mapping(address => uint256) invests;
    constructor() public{
        goal = 20 ether;
        invested = 0;
        owner = msg.sender;
    }
    function withdraw() public {
        if(phase == 1) {
            bug();  // There exists a bug
            owner.transfer(invested);
    } }
    function invest(uint256 amount) public payable {
        if (invested < goal){
            invests[msg.sender] += amount;
            invested += amount;
            phase = 0;
        } else { phase = 1; }
    }
    function refund() public {
        if (phase == 0) {
            msg.sender.transfer(invests[msg.sender]);
            invested -= invests[msg.sender];
            invests[msg.sender] = 0;
    } }
}
```

Figure 2: Fundraiser, a Loan System Solidity Code [37]

Smart Contract Fuzzing. Unlike traditional fuzzing that targets crashes, smart contract fuzzing must address state-dependent execution, as contract state relies on prior transaction sequences [15, 16]. Thus, its core task is generating and mutating transaction sequences to explore these state dependencies. Furthermore, vulnerabilities in smart contracts often stem from complex logic errors rather than runtime crashes. In this context, fuzzers typically integrate predefined oracles within the execution engine to detect specific flaw patterns like reentrancy or block dependency [20, 21, 49].

LLM for Fuzzing in Smart Contracts. Large Language Models (LLMs) [19, 55] are advanced AI systems with strong logical reasoning and text generation capabilities [52]. These capabilities enable their use in fuzzing frameworks [11, 41], where they enhance vulnerability detection by generating realistic inputs and contextually relevant transaction sequences that reflect genuine user interactions [43]. This overcomes limitations of traditional methods, which often rely on random or rule-based generation, lack contract-specific logic, and miss nuanced vulnerabilities [42]. Leveraging LLMs thus enables more sophisticated fuzzing techniques for accurate vulnerability detection, strengthening smart contract security.

3 Motivating Example

To further illustrate our idea, let us present a more detailed example. Consider the motivating example shown in Figure 2, a simplified version of the `Fundraiser`, which is a variant of [37] – abstracted from a real-world contract to capture its core functions and bug.

The `Fundraiser` contract consists of a `constructor` and three key functions: `withdraw`, `invest`, and `refund`. The `constructor` is executed only once when the contract is deployed. It sets the fundraiser

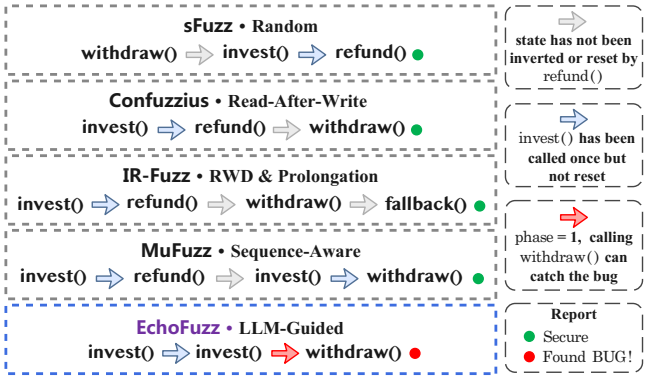


Figure 3: Different fuzzers generate sequences

goal to 20 ether (i.e., `goal` = 20 ether on line 8), initializes the invested amount to 0 (i.e., `invested` = 0 on line 9), and designates the contract creator as the owner (i.e., `owner` = `msg.sender` on line 10). Users can participate in the `Fundraiser` by calling `invest` while the contract is active (i.e., `phase` = 0). During this period, users can request a refund by invoking `refund` while the `Fundraiser` is ongoing (`phase` = 0). The `Fundraiser` concludes when the investment reaches or exceeds the goal (i.e., `invested` ≥ 20), at this point the contract owner can withdraw all funds by invoking the `withdraw` function. In this context, the function `withdraw` contains a bug at line 14 that can only be detected by the fuzzer if the branch condition satisfies `phase` = 1. Achieving this goal is by no means trivial, as it requires the fuzzer to generate executable transaction sequences while accounting for strict dependencies and the impact of other transactions, which is often quite challenging.

For instance, suppose that the fuzzer generates a sequence [`withdraw` → `invest`(∗) → `refund`], where ∗ can be any value, meaning all situations in the function `invest` can bring. However, due to a read-dependency of the state variable `phase` on `invest`, mutating inputs within this sequence cannot trigger the bug. To catch the bug, the fuzzer may need to generate a sequence such as [`invest`(∗) → `refund` → `withdraw`]. However, this sequence also fails to catch the bug, as calling `invest` once cannot enter the else-branch at line 22 to set `phase` = 1. To reach the else-branch, `invest` needs to be executed at least twice in the execution sequence, such as [`invest`(∗) → `refund` → `invest`(∗) → `withdraw`]. However, this sequence overlooks the impact from the function `refund` – once `refund` is called after `invest`, the state variable `invests[msg.sender]` resets to 0, making `invested` be reduced, as a result offsetting the effect of the previous `invest` call. Hence, sequences like [`invest`(∗) → `refund` → `invest`(∗) → `withdraw`] or [`invest`(∗) → `invest`(∗) → `refund` → `withdraw`] cannot set `phase` = 1, which fails to catch the bug at line 14. Furthermore, to find the critical sequence that catches the bug while reaching a minimized state space, the ideal sequence would be [`invest`(∗) → `invest`(∗) → `withdraw`]. Without calling the `refund` function, the first call meets the fundraiser goal, and the second enters the else-branch at line 22. Next, we show the limitations of existing fuzzers for identifying such a specific sequence, which we call the vulnerable function call sequence.

Limitations of Existing Fuzzers. We evaluate several state-of-the-art smart contract fuzzers to our example, including sFuzz [33], Confuzzius [45], IR-Fuzz [29], and MuFuzz [37]. None detect the bug at line 14, as they fail to generate sequences that enter the if-branch at line 13. In contrast, EchoFuzz reasons about the contract logic, entering the critical branch and exposing the bug in seconds. Existing fuzzers struggle to identify critical sequences and reach deep branches. As Figure 3 shows, they typically call all functions for completeness and order them via predefined rules.

Specifically, sFuzz uses a random generation strategy, making it difficult to identify meaningful sequences. Confuzzius utilizes the read-after-write technique and combines symbolic execution for dynamic data analysis to find potentially relatively meaningful sequences, but struggles to handle situations where a function must be called multiple times, as the example in Figure 2. IR-Fuzz employs static data flow analysis to prioritize and introduces a prolongation technique, but still struggles with calling the same function multiple times. MuFuzz focuses on mutating sequences and detects that `invested` < `goal` is the key to branching, but it fails to account for the impact of `refund` on the contract state, missing interaction cancellation, and thus the vulnerability.

These limitations indicate that existing fuzzers tend to generate smaller state spaces to ensure computation feasibility and fuzzing efficiency, thus preventing excessively large state spaces in large contracts, which complicates the analysis. This hinders the ability to reach deeper path branches, preventing the discovery of complex vulnerabilities (corresponding to the aforementioned *Challenge 1*). Furthermore, the mutation technique used by fuzzers is constrained by predefined rules. While aiming for universality in sequence mutations, they inevitably overlook the unique logical designs of specific contracts. As a result, this limitation obstructs effective navigation toward subsequent meaningful path branches, ultimately creating a frustrating bottleneck within the fuzzing process (corresponding to *Challenge 2*). In the following sections, we will introduce our innovative methodology to address these challenges.

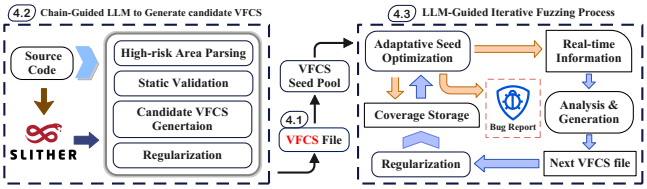


Figure 4: A high-level overview of EchoFuzz, including two key procedure: (1) Chain-guided LLM to Generate candidate VFCS and (2) LLM-guided Iterative Fuzzing Process.

4 Methodology

Overview. The overall architecture of our framework EchoFuzz is outlined in Figure 4. To address the above challenges, our framework shifts the fuzzing strategy from arbitrary function compositions to Vulnerable Function Call Sequences (VFCS). This approach is designed to:

- Balance vulnerability detection probability and exploration cost by generating minimal, logically meaningful sequences.
- Overcome fuzzing stagnation through LLM-guided iterative refinement of sequences.

We first formalize VFCS as the core abstraction for vulnerability-centric exploration, then detail how EchoFuzz generates and optimizes these sequences using LLMs. The `Fundraiser` contract (Figure 2) is used throughout this section to illustrate our method.

4.1 Vulnerable Function Call Sequence (VFCS)

Existing software engineering-based techniques, such as symbolic execution, static analysis and other rule-based mutation, face critical shortcomings in smart contract fuzzing: (1) Logical Gap in State Transitions. While they can track inter-function data dependencies (e.g., invested is read in withdraw), they fail to recognize functional relationships like *offsetting interactions*. For example, in the `Fundraiser` contract, `refund` resets `invested`, neutralizing the cumulative effect of prior `invest` calls – a nuance overlooked by rule-based approaches. (2) Combinatorial Redundancy. Tools like MuFuzz generate redundant sequences (e.g., [`invest` → `refund` → `invest` → `withdraw`]) to further increase the probability of discovering vulnerabilities, significantly increasing computational costs compared to minimal paths. These shortcomings exemplify the fundamental tension in *Challenge 1*, i.e. achieve a high detection probability without prohibitive exploration costs.

To address these limitations, we propose **Vulnerable Function Call Sequence (VFCS)** as a logic-aware paradigm. VFCS tackles these issues by formalizing minimal critical paths that are directly responsible for triggering vulnerabilities. A VFCS is defined as a sequence $v = [f_{v_1}, f_{v_2}, \ldots, f_{v_k}]$ with three essential properties:

1. Vulnerability Trigger: The final call f_{v_k} executes under a state $\mathcal{S}_k$ that exposes the vulnerability.

2. Dependency Chain: Each $f_{v_i} (i < k)$ alters $\mathcal{S}$ to enable $f_{v_{i+1}}$. Formally:

$$\mathcal{S}_i \xrightarrow{f_{v_i}} \mathcal{S}_{i+1} \quad \text{where} \quad \mathcal{S}_{i+1} \models \text{Pre}(f_{v_{i+1}})$$

3. Minimality: No proper subsequence of v can trigger the bug.

The formula $\mathcal{S}_i \xrightarrow{f_{v_i}} \mathcal{S}_{i+1}$ represents the transition of the system state $\mathcal{S}$ after executing function f_{v_i}, where $\mathcal{S}_{i+1}$ satisfies the preconditions for the next function call. The symbol $\models$ means "satisfies", indicating that the new system state $\mathcal{S}_{i+1}$ meets the preconditions for the next function call $f_{v_{i+1}}$.

For instance, the sequence [`invest` → `invest` → `withdraw`] in motivating example is a VFCS, because: (1) `withdraw` executes when `phase = 1`, triggering the bug; (2) For dependency chain, first `invest`, sets `invested = donations`. Second `invest`, pushes `invested ≥ goal`, transitioning `phase = 1`; (3) Minimality: Inserting `refund` or removing either `invest` breaks the chain.

However, directly generating such sequences is impractical, and satisfying all VFCS properties simultaneously is idealized. Without prior knowledge of vulnerability locations and exact trigger conditions, simultaneously ensuring minimality, maintaining dependency chains, and triggering specific vulnerabilities is impractical in practice. Hence, we pursue VFCS candidates step by step that can approximate three properties as closely as possible, thereby reducing the gap between the candidate VFCS and the ideal concept.

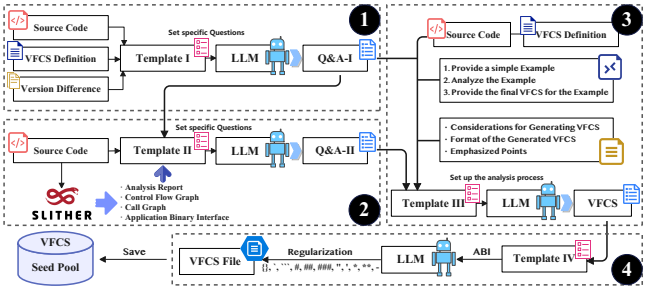


Figure 5: Chain-guided LLM Sequence Generation Process

4.2 Chain-Guided LLM to Generate Candidate VFCS

To bridge this gap, we take advantage of the code comprehension capabilities of LLMs to generate candidate VFCS that better approximate these ideal properties.

How can we guide LLMs to achieve this goal? To answer this, we abstract the workflow of expert contract auditing to develop a structured approach for sequence extraction, consisting of three key steps: *(1) High-risk Area Identification.* Identifying critical functions and variables, taking into account potential state alterations due to function offsetting relationships (e.g., `invest` and `refund` in `Fundraiser` which modify shared state variables like `phase`). *(2) Contextual Validation.* Validating hypotheses using static analysis tools (e.g., control flow graphs from *Slither*) to establish execution constraints. *(3) Pattern Matching.* Leveraging known vulnerability patterns to guide scenario construction (e.g., identifying potential bugs in `withdraw`).

Inspired by their methodology, we utilize LLMs to construct a chain-guided analysis procedure as shown in Figure 5, which allows us to logically generate candidate VFCS. Next, we will describe each step in detail, and we have added a regularization module at the end to improve the availability of candidate VFCS, the effects of which are discussed in Section 5.4.

Step 1: High-risk Area Parsing. The LLM processes the contract source code via Template-I to extract critical functions and variables, while identifying potential offsetting function call relationships. It utilizes the concept of VFCS, which is provided as part of the input, to guide this analysis. Additionally, the model references a document detailing impossible vulnerability types for different contract versions to reduce model hallucinations. For example, in Figure 2, it identifies `invest` and `withdraw` as high-risk functions in the `Fundraiser` contract, and recognizes the offsetting relationship between `invest` and `refund`, highlighting the need to carefully examine their calling patterns.

Step 2: Static Validation. By integrating the contract source code with the static analysis tool *Slither*, the LLM extracts precise function call information and control flow data. Using static analysis to enumerate function call combinations, the LLM validates the high-risk area parsing results from Step 1 and identifies potential function combinations that could trigger state changes or introduce vulnerability risks. For example, through enumerated call relationships, it identifies that a call sequence [`invest` → `invest`] under specific parameters would alter the `phase` variable state, effectively linking critical state variable changes to consecutive `invest` calls.

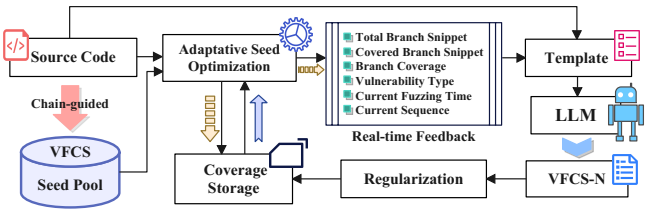


Figure 6: Feedback-Driven LLM-guided Iterative Fuzzing Process

Step 3: Candidate VFCS Generation. Building upon the results of Step 1&2, the LLM re-examines the source code and demonstrates the process of deriving VFCS from a sample contract using the gathered analytical and validation information. It outlines key points that need emphasis in VFCS generation, ensuring alignment with the ideal VFCS concept throughout. For example, by integrating reports from Step 1&2, the LLM identifies [`invest` → `invest` → `withdraw`] as a VFCS through the rationale that first `invest` sets `invested>0`, second `invest` triggers `phase=1`, and `withdraw` triggers the vulnerability, representing the candidate VFCS to be generated.

Step 4: Regularization. In the final step, the LLM generates an ABI-compatible VFCS JSON file based on the ABI and the candidate VFCS. Automated scripts clean the file, ensuring it complies with ABI format and JSON standards. The final verified files are stored in the seed pool for fuzzing, ready for automated vulnerability testing.

The chain-guided approach analyzes contract-specific logic and inter-function interactions to generate candidate VFCSs that target vulnerability detection. Unlike prior LLM-driven methods that focus on code prioritization and input generation, our approach generates VFCS candidates that are logically aware of state transitions within contracts (for more details, see Section 7).

4.3 LLM-Guided Iterative Fuzzing Process

In practical applications, contract state transitions are often obscured by complex parameter constraints, requiring specific transaction sequences to reach branches controlled by intricate variables. Existing fuzzers encounter two main limitations: (1) they either fail to alter the transaction sequence, repeating tests on the same paths, thereby hindering deep branching exploration, or (2) they modify the sequence within rigid predefined rules, leading to repetitive fragments and limited access to targeted branches. These issues create a bottleneck in fuzzing, especially when navigating deep, nested conditions where critical branching points reside.

To tackle this challenge, we propose a framework based on an LLM-guided iterative fuzzing process, designed to dynamically refine sequence generation through iterative feedback. The process starts by selecting a candidate VFCS from the *seed pool*, generated through a chain-guided procedure (see Section 4.2), and using it in fuzzing to extract real-time feedback specific to the contract. This feedback includes key data such as *Total Branch Snippet*, *Covered Branch Snippet*, *Branch Coverage*, identified *Vulnerability Type*, *Current Fuzzing Time*, and the *Current Sequence*. The feedback is then embedded into a predefined template, which prompts the LLM to generate new contract-focused sequences aimed at reaching deeper, unexplored branches. After each fuzzing iteration, the real-time feedback and coverage data are stored as part of the *Coverage Storage*. Once the LLM produces a new candidate VFCS, the saved coverage is reloaded, and the next fuzzing round begins. This iterative process creates a continuous loop of coverage storage, feedback generation, and sequence optimization. Figure 6 presents a detailed visualization of each step in this iterative process, illustrating how sequence generation evolves based on both contract properties and fuzzing outcomes. Throughout the fuzzing process, we apply branch distance-based seed mutation to guide the direction of seed mutation, continuously adapting and optimizing as reflected in the *Adaptive Seed Optimization* step in the diagram.

By integrating the LLM-guided approach, our method expands the sequence range and introduces logical understanding to the mutation process. This allows the fuzzer to generate more meaningful mutations, crucial for complex smart contracts with deep branching conditions that simple random mutations often overlook.

Adaptive Seed Optimization. For function parameter generation, EchoFuzz follows the same proven strategies as state-of-the-art tools like IR-Fuzz and MuFuzz, using randomly generated seeds with branch distance-guided mutations towards unexplored paths. Each iteration refines the mutations, optimizing seeds for deeper exploration of complex branching structures. This adaptive strategy enhances coverage of critical code paths, ensuring the fuzzer efficiently uncovers hidden vulnerabilities and improves overall effectiveness in identifying potential security flaws.

5 Experiment

In this section, we conduct extensive experiments to evaluate EchoFuzz with the aim of answering the following research questions:

- **RQ1.** Does EchoFuzz demonstrate superior performance compared to state-of-the-art tools?
- **RQ2.** What are the contributions of different components within EchoFuzz?
- **RQ3.** What is the impact of different LLMs on VFCS performance, and which LLM performs best?

In the following, we first introduce the experiment settings, followed by answering the above questions one by one. Additionally, we provide a case study to illustrate our workflow.

5.1 Experiment Settings

5.1.1 Implementation. EchoFuzz is composed of over 2,000 lines of Python code, 4,800 words of prompts and 6,000 lines of C++ code. We implement EchoFuzz on the basis of IR-Fuzz [29] (a state-of-the-art smart contract fuzzer). For each component, our templates follow a unified structure: (1) Source & intermediate results, (2) Analysis methodology, (3) Analysis objectives and outputs. Each step uses specific prompts to guide LLM analysis, including High-risk Area Parsing (Template-I), Static Validation (Template-II), Candidate VFCS Generation (Template-III), and Iterative Refinement (Template-Sec 4.3). All detailed templates are available at: https://github.com/fuzzersc/vulnerabilities/tree/main/templates. Our EchoFuzz framework, including all prompt templates and related tools, is open-sourced on GitHub.

5.1.2 Dataset. Following prior work [13, 29, 45], we extend existing datasets, resulting in three distinct datasets: **D1**: A mixed dataset of 202 medium and large contract projects, randomly selected from

Table 1: Statistical Overview of Evaluated Datasets.

Avg. Metric in Dataset (Count)	D1 (202)	D2 (143)	D3 (145)
Contract Size (KB)	3.49	3.44	7.58
Functions per Contract	8.31	6.44	22.46
Parameters per Function	1.33	1.17	1.73
State Variables per Contract	5.23	4.99	6.74
Lines of Code per Contract	92.78	84.72	192.49

prior studies for diversity and cost control, as multiple LLMs, including high-cost, closed-source models, are used in our testing. **D2**: A dataset of 143 contract projects with labeled vulnerabilities from previous research [13]. This collection provides a benchmark for evaluating the vulnerability detection capabilities of our method. **D3**: A data set of 146 large and complex contract projects that we manually scraped from Etherscan [12]. These on-chain contracts add a real-world element to our analysis, enabling us to test our approach on live, previously unseen contracts.

Table 1 presents the statistical overview of these three datasets. Dataset D3 represents the largest contracts, with 7.58KB average size and 22.46 functions per contract. Given that contracts include extensive comments (helpful for LLM understanding), we classify contract-scale using source file size rather than bytecode instructions: Medium-scale (1-6KB), Large-scale (>6KB). We focus on medium- and large-scale contracts, as existing tools handle <1KB contracts effectively [13, 14, 24], and the performance on <1KB contracts is further discussed in Section 6. This classification aligns with prior work [18, 29, 37] that employs similar scale distinctions based on instruction count, typically using 3,000 instructions as the threshold for large-scale contracts, as shown in Table 6.

Table 2: Overview of selected LLMs and their advantages. In Type column, O: Open-Source; C: Closed-Source. In Usage column, LD: Local Deployment.

LLMs	Type	Available URL	Advantages	Usage
Qwen2-7B	O	https://ollama.com/library/qwen	Widely Used lightweight LLM in Chinese	LD
Llama3.1-8B	O	https://ollama.com/library/llama3.1	Widely Used lightweight LLM	LD
Llama3.1-405B	O	https://ollama.com/library/llama3.1	Strongest Open-source LLM	API
GPT 3.5 Turbo	C	https://platform.openai.com/	Long Time Used and Cost-Effective LLM	API
GPT 4o-mini	C	https://platform.openai.com/	Widely Used, Highly Cost-Effective LLM	API
Claude3.5 Sonnet	C	https://docs.anthropic.com/	Strongest Closed-source LLM currently	API

5.1.3 Baseline. To compare the performance and effectiveness of EchoFuzz, we consider both the LLM and the Fuzzer aspects.

LLM. Table 2 compares the LLMs selected, emphasizing their characteristics and selection rationale. Our diverse choices cover various use cases, including lightweight open-source models like Qwen2-7B and LLaMA3.1-8B for local deployment, and the LLaMA3.1-405B as the strongest open-source model. To ensure cost-effectiveness, we also include the widely used GPT 4o-mini and GPT 3.5 Turbo for API deployment. Lastly, we selected Claude 3.5 Sonnet, the most powerful closed-source model, to explore peak performance. This varied selection allows for a thorough evaluation of our framework's usability and performance across different model types.

Fuzzer. We include sFuzz [33], IR-Fuzz [29], and MuFuzz [37] in our comparisons for two reasons: (1) Their performance has been experimentally proven to surpass other smart contract fuzzing tools, such as ContractFuzzer [24], Echidna [14], ContraMaster [8, 47], ILF [18], and Confuzzius [45]; and (2) these tools are commonly used as benchmarks in current research. Currently, MuFuzz is considered the state-of-the-art fuzzer. We do not compare with LLM4Fuzz [41] as it lacks open-source availability and accessible artifacts. A technical comparisons are provided in Section 7. We compare EchoFuzz against these fuzzers in terms of branch coverage and vulnerability detection. For each fuzzing process, we perform fuzzing with all three generated VFCS candidates and take the average results for comparison. When 60s elapses without discovering new paths, the fuzzing process transitions from the initial VFCS to an LLM-driven iterative process, designed to explore more efficient paths.

Device. We conduct our evaluations on an Ubuntu 20.04 system equipped with an Intel(R) Xeon(R) Platinum 8255C CPU (12 virtual cores, 2.50GHz), 40GB RAM, and a 10GB RTX 3080 GPU.

5.2 Performance Evaluation (RQ1)

We employ Claude 3.5 Sonnet as the LLM in our experiments, which has been proven to be the strongest among selected LLMs in Section 5.4. Subsequently, we compare the performance of EchoFuzz, which is implemented following the workflow shown in Figure 4, with several other fuzzers. The comparison is based on two metrics: *vulnerability detection* and *branch coverage*.

5.2.1 Vulnerability Detection. We conduct vulnerability detection in datasets D1, D2, and D3 using EchoFuzz, in conjunction with state-of-the-art fuzzing tools. Notably, we adopt a strict oracle standard that is consistent with prior works such as IR-Fuzz and MuFuzz. Given that fuzzing typically reveals true vulnerabilities and that previous studies have thoroughly analyzed false positives and false negatives under this oracle standard [29, 37], we focus on carefully evaluating EchoFuzz's effectiveness in identifying vulnerabilities across different datasets and understanding their distribution more comprehensively.

The results, presented in Table 3 in the format IR-Fuzz/MuFuzz/EchoFuzz, demonstrate that EchoFuzz consistently outperforms state-of-the-art fuzzers across various datasets and vulnerability types. Specifically, EchoFuzz detects 48.00%, 42.55%, and 146.67% more vulnerabilities than MuFuzz on D1, D2, and D3, respectively, with an overall average improvement of 62.77%. These results highlight the substantial advantages of EchoFuzz over existing tools in various scenarios, particularly when analyzing new, large-scale, and complex real-world contract projects on the blockchain (dataset D3). As contract structures become increasingly complex, traditional approaches that rely solely on syntactic patterns struggle to effectively uncover meaningful attack sequences. We attribute the superior performance of EchoFuzz to its logic-driven approach, which extracts VFCS to optimize state transitions and takes advantage of real-time feedback to guide the LLM in exploring contract logic and simulating attacks that exploit the unique logic of each contract. Encouragingly, we have discovered 37 previously unknown vulnerabilities from 19 newly contract projects, whereas other state-of-the-art tools could only identify 13 and 15 vulnerabilities, respectively. The details are put in anonymous URL: https://github.com/fuzzersc/vulnerabilities.

False Negative Analysis. We conduct a detailed review of the two false negatives of EchoFuzz compared to MuFuzz on GL, shown in Table 3. The review reveals that the issue stems from insufficient cumulative iteration times of the seeds and the limitations of seed exploration in EchoFuzz. In the context of simple contracts, the

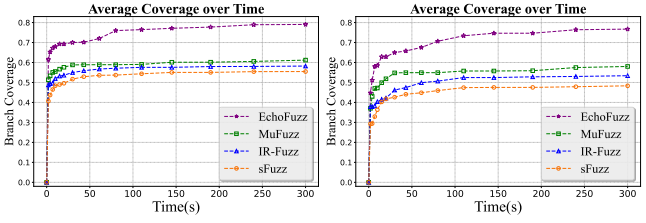
Table 3: Results show by format IR-Fuzz/MuFuzz/EchoFuzz, respectively. EchoFuzz against state-of-the-art fuzzers in Vulnerability Detection of Different Dataset. In Oracle setting, GL: Gasless, IO: Integer Over-/under-flow, RE: Reentrancy, UC: Unchecked Call, BN: Block Number Dependency, TP: Timestamp Dependency, DG: Dangerous Delegatecall, UE: Unexpected Ether or Ether Frozen. In ET row, it means Each Type.

Dataset	GL	IO	RE	UC	BN	TP	DG	UE	Each Dataset
D1	23/26/**31**	60/39/**75**	17/19/**22**	10/12/**13**	7/8/**12**	13/15/**22**	1/1/**3**	5/5/**7**	136/125/**185** (***48.00%*** ⇑)
D2	22/**27**/25	32/20/**53**	19/18/**20**	14/14/**15**	4/5/**8**	4/5/**7**	2/2/2	3/3/**4**	100/94/**134** (***42.55%*** ⇑)
D3	2/2/**8**	8/6/**15**	2/2/**7**	1/1/1	0/0/0	1/1/**3**	1/1/1	0/0/**2**	15/13/**37** (***146.67%*** ⇑)
ET	47/55/**64**	100/65/**143**	38/39/**49**	25/27/**29**	11/13/**20**	18/23/**32**	4/4/**6**	8/8/**13**	250/231/**376** (***62.77%*** ⇑)

function call sequences are relatively fixed. Compared to EchoFuzz, MuFuzz follows rule-based mutation, can maintain a simple sequence while spending more time mutating seeds, allowing seeds to reach and vary in a wider range, and thus reach deeper branches. This represents a limitation of our approach, which we discuss in more detail in the seed mutation part of Section 6.

5.2.2 Branch Coverage. We evaluate branch coverage of EchoFuzz, sFuzz, IR-Fuzz, and MuFuzz on medium and large contracts in D1. EchoFuzz consistently outperforms all others, as shown in Figure 7. On medium contracts, EchoFuzz achieves 79.1% coverage, surpassing sFuzz, IR-Fuzz, and MuFuzz by 38.40%, 35.80%, and 29.19%, respectively (Figure 7(a)). On large contracts (Figure 7(b)), all tools show lower coverage, but EchoFuzz's decline is minimal (2.4%). The declines of others are 3.76×, 2.07×, and 1.36× greater. EchoFuzz still achieves 76.70% coverage, outperforming others by 58.75%, 43.71%, and 32.24%, demonstrating superior robustness. EchoFuzz outperforms other fuzzers on large contracts, achieving higher coverage rates in less time. This is due to its chain-guided VFCS generation, which enhances testcase quality beyond that of fuzzers relying on Random and Read-Write dependency strategies.

Furthermore, EchoFuzz continues to increase coverage effectively, while other fuzzers typically see their performance gains rapidly diminish after 60s, quickly hitting fuzzing bottlenecks that are hard to overcome. Beyond this, limited state space sharply reduces effectiveness, yielding diminishing returns with minimal further improvement. Further analysis reveals that the coverage improvement is more substantial for large contracts. EchoFuzz achieves a 10.02% improvement on medium contracts, while other fuzzers only improve by around 3%, demonstrating a 3× performance boost. This suggests that, after a period without discovering new paths, the fuzzing process transitions from the initial VFCS to an LLM-driven iterative approach. In this phase, fuzzers are no longer limited by their fixed state space. Even MuFuzz, which employs sequence-aware mutation strategies, struggles to explore new branches effectively. In contrast, EchoFuzz continues to demonstrate impactful mutations. This is due to LLM-guided VFCS generation: leveraging logic insights and real-time feedback, the LLM analyzes covered code and generates targeted tests for unexplored branches, enhancing mutation effectiveness.



(a) On Medium contracts **(b) On Large contracts**

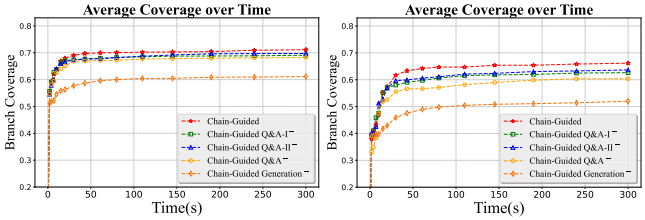
Figure 7: Branch coverage comparison between EchoFuzz and the state-of-the-art fuzzers.

Answer to RQ1: EchoFuzz outperforms state-of-the-art tools in both branch coverage and vulnerability detection. On average, it achieves 29% increase in coverage and 62% improvement in vulnerability detection, particularly in large contracts.

5.3 Ablation Study (RQ2)

By default, EchoFuzz uses a chain-guided LLM to generate high-quality candidate VFCSs for fuzzing. We are curious about how each component within the chain-guided procedure contributes to the overall performance. Furthermore, it is interesting to observe the effects of removing the entire chain-guided strategy. We seek to understand how the two core LLM-utilizing approaches compare against pure static analysis to isolate the LLM's contribution. Additionally, EchoFuzz uses real-time fuzzing feedback to guide the LLM in generating the new VFCS for the fuzzing process. We are also curious to see how much this method contributes to the overall performance of EchoFuzz. Next, we conduct experiments on each of these key components separately.

5.3.1 Chain-guided Framework Component Analysis. We conduct ablation experiments by removing components: Q&A-I (High-risk Area Parsing), Q&A-II (Static Validation), both Q&A steps (Q&A$^-$), and Candidate VFCS Generation (Generation$^-$). Figure 8 presents the branch coverage performance comparison. The results shows that removing any component degrades performance, with escalating impact on larger contracts. Individual Q&A step removal causes 2.96%-5.66% coverage loss, while their collective removal (Q&A$^-$) yields 4.26%-9.63% degradation, demonstrating synergistic effects. Most critically, Generation$^-$ causes severe 16.37%-26.41% degradation, as this component synthesizes Q&A insights into actionable VFCS candidates—without it, the framework reduces to dependency methods like existing tools. These results validate the effectiveness of each component, particularly for large-scale contracts where multi-step reasoning proves essential.



(a) On Medium contracts **(b) On Large contracts**

Figure 8: Ablation study of chain-guided LLM procedure in EchoFuzz on branch coverage.

Table 4: EchoFuzz each components in Vulnerability Detection of Different Dataset. Relevant oracle setting can see Table 3. Results show by format EchoFuzz-RD/EchoFuzz-SA/EchoFuzz-WCG/EchoFuzz-WIP/EchoFuzz, respectively.

Dataset	GL	IO	RE	UC	BN	TP	DG	UE	Each Dataset
D1	12/23/26/27/**31**	47/60/66/69/**75**	8/17/19/21/**22**	6/10/11/12/**13**	7/7/9/11/**12**	11/13/15/18/**22**	1/1/3/3/3	5/5/6/6/**7**	97/136/155/167/**185**
D2	16/22/24/20/**25**	30/32/45/41/**53**	10/19/19/16/**20**	8/14/14/13/**15**	4/4/6/5/**8**	5/4/6/5/**7**	1/2/2/2/2	3/3/3/3/**4**	77/100/119/105/**134**
D3	1/2/7/6/**8**	8/8/11/9/**15**	1/2/6/6/**7**	0/1/1/0/1	0/0/0/0/0	1/1/1/1/**3**	0/1/1/1/1	0/0/1/1/**2**	11/15/28/24/**37**
ET	29/47/57/53/**64**	85/100/122/119/**143**	19/38/44/43/**49**	14/25/26/25/**29**	11/11/15/16/**20**	17/18/22/24/**32**	2/4/6/6/6	8/8/10/10/**13**	185/250/303/306/**376**

5.3.2 Study on Chain-guided Generating Initial VFCS. We replace chain-guided LLM process in EchoFuzz with random sequence generation, creating **EchoFuzz-WCG**("**W**ithout **C**hain-guided **G**eneration"). This variant samples functions randomly based on their distribution in the contract code. As Figure 9 shows, EchoFuzz-WCG initially lags behind EchoFuzz-SA in the first 60s due to the absence of targeted guidance. However, it later surpasses EchoFuzz-SA by leveraging LLM-guided iterations. In contrast, the full chain-guided version (EchoFuzz-WIP) outperforms EchoFuzz-SA immediately, achieving 28% higher coverage in the first few seconds.e At 300s, EchoFuzz achieves 6.98% higher coverage than EchoFuzz-WCG and identifies 24.09% more vulnerabilities on average. The chain-guided LLM procedure provides immediate effectiveness through multi-step reasoning and high-quality initial VFCS generation, removing random exploration phases and enabling targeted vulnerability discovery from the start of fuzzing.

5.3.3 Study on LLM-guided Iteration Process. We remove the LLM-guided iterative fuzzing process from EchoFuzz, conducting fuzzing with only the original VFCS without further modifications, resulting in **EchoFuzz-WIP** ("**W**ithout LLM-guided **I**teration **P**rocess"). This variant employs the chain-guided LLM procedure but lacks real-time feedback-driven sequence refinement. As shown in Figure 9 and Table 4, EchoFuzz-WIP demonstrates immediate effectiveness through chain-guided generation, achieving 28% higher coverage than EchoFuzz-SA in the first few seconds and significantly outperforming EchoFuzz-WCG in beginning. However, the complete EchoFuzz framework shows a 10.65% coverage improvement over EchoFuzz-WIP at 300s. In vulnerability detection, particularly in complex contracts (D3), EchoFuzz identifies 54.17% more vulnerabilities than EchoFuzz-WIP. LLM-guided iteration process enables EchoFuzz to overcome the exploration bottleneck faced by other fuzzers between 60s-300s, achieving 22.06% coverage improvement (vs. 2-5% for other fuzzers). This nearly order-of-magnitude enhancement over existing tools is extremely encouraging.

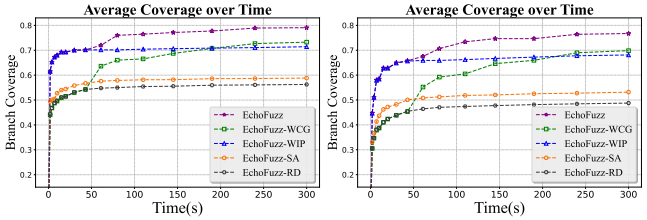
5.3.4 LLM Contribution Analysis: Comparison with Non-LLM Baselines. To isolate the LLM's contribution, we introduce two non-LLM baselines: **EchoFuzz-RD** ("**R**an**d**om") and **EchoFuzz-SA** ("**S**tatic **A**nalysis only"), enabling direct comparison with our LLM-based approaches EchoFuzz-WIP and EchoFuzz-WCG. As shown in Figure 9 and Table 4, both LLM approaches achieve substantial improvements over non-LLM baselines. In coverage, they demonstrate more than 23% and 29% improvements over EchoFuzz-SA and EchoFuzz-RD respectively. In vulnerability detection, both achieve over 21% and 63% improvements over the non-LLM baselines respectively. The complete EchoFuzz framework achieves 50.4% and 103.2% vulnerability detection improvements over EchoFuzz-SA and EchoFuzz-RD respectively, validating the significant contribution of LLM-based guidance in smart contract fuzzing.

Answer to RQ2: Each component in the chain-guided framework is essential. EchoFuzz achieves 50.4% improvement over static analysis alone, while the LLM-guided iteration process provides 22% coverage improvement between 60s-300s, showing the significant contribution of LLM-based approaches.

5.4 VFCS Performance with Different LLMs (RQ3)

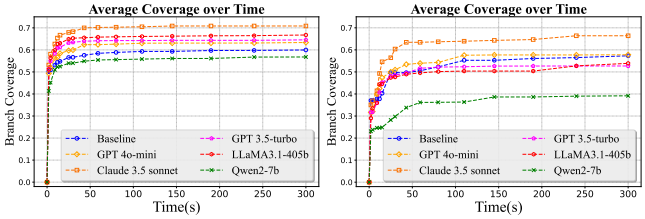
We evaluate VFCS performance across different LLMs using two key metrics: *branch coverage* and *VFCS available ratio*. These metrics comprehensively assess both the quality and applicability of generated test cases across our contract dataset.

5.4.1 Branch Coverage in Different LLMs. We evaluate effectiveness of VFCS by measuring branch coverage in D1, comparing various LLMs with the baseline (IR-Fuzz) as shown in Figure 10. Figure 10(a) shows results for medium contracts, and Figure 10(b) for large contracts, illustrating coverage trends over time. Claude 3.5 Sonnet consistently outperforms both other models and the baseline once fuzzing stabilizes, regardless of contract size. On medium contracts, it achieves 70.84% branch coverage, an impressive 18.14% higher than the baseline. Other models, such as LLaMA3.1-405B, GPT 3.5-turbo, and GPT 4o-mini, show smaller gains of 11.32%, 7.66%, and 5.66%, respectively, while Qwen2-7B underperforms with 5.31% lower coverage than the baseline. On large contracts, Claude 3.5 Sonnet again leads, achieving a remarkable 15.85% higher branch coverage than the baseline. GPT 4o-mini provides a modest improvement of 0.85%, while LLaMA3.1-405B, GPT 3.5-turbo, and Qwen2-7B fall below the baseline by 5.98%, 8.06%, and 31.66%, respectively. For



(a) On Medium contracts (b) On Large contracts

Figure 9: Ablation study of EchoFuzz.



(a) On Medium contracts (b) On Large contracts

Figure 10: Different LLMs generation VFCS in Branch coverage.

Table 5: Performance of Different LLMs on VFCS Available Ratio and Regularization Improvement.

LLM Version	Quantity-NoR	Quantity	Improve	Available Ratio
Qwen2-7B	263	401	52.5%	66.2%
LLaMA3.1-8B	182	255	40.1%	42.1%
LLaMA3.1-405B	281	541	**92.5%**	89.3%
GPT 3.5 Turbo	**385**	557	44.7%	91.9%
GPT 4o Mini	345	549	59.1%	90.6%
Claude 3.5 Sonnet	379	**571**	50.7%	**94.2%**

larger contracts, LLMs tend to prioritize vulnerability-triggering paths over broader branch coverage, limiting their effectiveness. This suggests the need for LLM-guided iteration combined with additional fuzzing to maximize coverage and uncover hidden paths. These results highlight model size impact and closed-source models' advantages for VFCS. Smaller models decline in effectiveness with larger contracts. Claude 3.5 Sonnet demonstrates superior performance by achieving high branch coverage quickly, making it the preferred model for high-quality test case generation.

5.4.2 Available Ratio of VFCS Generated by Different LLMs. To evaluate the availability of VFCS generated by different LLMs, we designed our framework to generate three VFCS candidates per contract in dataset D1, which contains 202 contracts, targeting a total of 606 VFCS files. Initial experiments revealed that a significant portion of generated VFCS were unusable, leading to resource waste. By integrating a regularization module, our framework significantly improve VFCS available ratio. In Table 5, "NoR" refers "No-use-Regularization," with *Quantity-NoR* as usable VFCS without regularization. *Improve* reflects the availability gain from regularization, and *Available Ratio* is usable VFCS relative to the target of 606. As shown in Table 5, LLaMA3.1-405B's VFCS availability increases from 281 to 541, an increase of 92.5%. Other LLMs also exhibit roughly a 50% increase in availability with regularization. In our framework, Claude 3.5 Sonnet achieves the highest availability ratio at 94.2%, indicating that our approach is both practical and versatile across a wide range of contract types. Similarly, GPT 3.5 Turbo and GPT 4o-mini both surpass the 90% threshold, while LLaMA3.1 405B approaches 90%, confirming the strong performance of larger models within our framework. In contrast, smaller models such as LLaMA3.1 8B achieve a notably lower availability ratio of 42.1%, highlighting their limitations in domain-specific tasks without specialized fine-tuning. This points to a promising direction for future research: enhancing the performance of smaller, more accessible models via targeted training techniques.

Answer to RQ3: Different LLMs influence VFCS performance, with Claude 3.5 Sonnet outperforming the baseline in branch coverage and exceeding 90% in availability, highlighting the effectiveness and availability of our framework.

5.5 Case Study

To illustrate how EchoFuzz applies its two core procedures for targeted sequence generation in contract logic to detect complex vulnerabilities and overcome exploration bottlenecks, we present its workflow via a case study. Figure 2 presents the contract, with detailed explanations on how to trigger the bug provided in Section 3. Other tools fail to detect this bug as they overlook the impact of `refund` on the state variable `invested`, achieving only 50% coverage. In contrast, EchoFuzz generates a sequence capable of triggering the bug using chain-guided procedure (i.e., EchoCV in Figure 1, with detailed procedure on how to generate that VFCS in Section 4.2) to generate *T1*: [`invest` → `invest` → `withdraw`]. After the first round of fuzzing, the bug is triggered at line 28. At this point, the branch coverage reaches 75%, as the *T1* sequence does not call `refund`. In the LLM-guided iterative process, the LLM identifies that all branches of the `invest` and `withdraw` are covered, and a potential bug is found within `withdraw`. Based on real-time feedback, the LLM generates a new sequence to explore branches in `refund` (EchoCV-N in Figure 1): *T2* = [`invest` → `refund`]. *T2* incorporates the coverage information saved at the end of the first round of fuzzing to initiate the second round of fuzzing. The second round of fuzzing ended in under a second, with branch coverage reaching 100%, completing the process. This workflow demonstrates that EchoFuzz ensures in-depth exploration of contract vulnerabilities while maximizing exploration of the contract itself.

6 Discussion

In this section, we discuss the limitations and potential improvements of EchoFuzz.

LLM Generation Analysis. The generation of EchoFuzz sequences depends on LLMs, introducing three key considerations:

Contract Scale Limitations. For <1KB contracts (~2 functions), all fuzzers can achieve high coverage (90%+), as optimal sequences are simple and shallow (excluding random strategy). Large contract sizes may exceed LLM context limits, harming our framework's usability. We conduct quantitative experiments in three datasets to examine how the contract size affects LLM response success rates. As shown in Table 6, success rates remain high (92%+) for contracts up to 50KB (12,000 tokens), but drop significantly to 75% for contracts exceeding 70KB (22,000 tokens) due to context limits. This indicates the need for contract segmentation strategies, hierarchical analysis approaches, or integration with more capable LLM architectures to handle enterprise-scale contracts.

LLM Overhead Analysis. LLM-based sequence generation introduces computational overhead that impacts overall fuzzing efficiency. We measured execution times across our experimental datasets comparing EchoFuzz against baseline approaches as shown in Table 7. While EchoFuzz requires 838s compared to MuFuzz's 603s, the LLM overhead accounts for only 28% of total execution time, yet achieves 1.6× improvement in vulnerability detection (376 vs. 231). Future optimizations could explore lightweight LLM architectures, caching mechanisms, or parallel processing strategies to reduce latency while maintaining generation quality.

LLM Effectiveness Conditions. LLM advantages over static analysis emerge when the model's performance sufficiently handles contract complexity. As demonstrated in Figure 10, in medium-scale contracts, four out of five LLMs notably outperform static analysis

Table 6: Contract size with its mean number of instructions impacts the success rate of LLM response

Contract Size	<6KB	~10KB	10-30KB	30-50KB	50-70KB	>70KB
Token Size	~1,150	~2,450	~4,200	~9,000	~12,000	>22,000
Instructions	~3,000	~4,500	~7,000	~15,000	~22,000	>45,000
Success Rate	99%	99%	94%	92%	83%	75%

Table 7: Average time overhead for fuzzers to test one contract. IR-Fuzz, MuFuzz and EchoFuzz all have pre-fuzz and main-fuzz stages.

Fuzzer	Overhead of Each Step (s)	Total (s)
sFuzz	Main: 300	300
IR-Fuzz	Setup: 2 + Pre: 300 + Main: 300	602
MuFuzz	Setup: 3 + Pre: 300 + Main: 300	603
EchoFuzz	Chain-guided: 76 + Pre: 300 + Iteration: 162 + Main: 300	838

baselines, while in large-scale contracts, only two LLMs demonstrate superior performance. When LLM capabilities are adequate, they provide distinct clear advantages over pure static analysis methods. Future work could focus on pre-training or fine-tuning more specialized LLMs to enhance their proficiency specifically for smart contract domains.

Data Dependency Analysis. EchoFuzz uses LLMs to generate sequences, different from traditional approaches. However, current data dependency analysis relies on traditional methods, potentially missing nuanced LLM-generated sequence dependencies. Additionally, function parameter selection corresponds to seed in the fuzzing process, as these parameters serve as mutational targets. Effective parameter combinations that trigger new paths or vulnerabilities are retained as interesting seeds for subsequent iterations, making parameter selection decisive in vulnerability discovery. Sequence switching timing affects parameter mutation adequacy, where appropriate timing ensures sufficient parameter space exploration. Future work could integrate LLMs and additional properties to guide sequence switching timing.

Sequence Diversity Analysis. EchoFuzz's current reliance solely on fuzzing time and new path discovery to determine sequence switching, that not only constrains sequence diversity but also leads to incomplete vulnerability exploration, resulting in false negatives. To address this, we explore Linear Time Temporal Logic (LTL) properties to enhance sequence diversity with state transitions and access control constraints, where state transitions enforce diverse function injection under temporal conditions, and access control extends switching thresholds based on exploration sufficiency. Experimental validation on False Negative cases from Section 5.2 shows that EchoFuzz+LTL resolves 2/2 false negative vulnerabilities with a 44% increase in unique sequence generation (from 9 to 13), as demonstrated in Table 8. Currently, LTL properties are manually generated based on contract analysis, and future work could automate the extraction of temporal constraints from contract code.

Table 8: Performance Improvement with LTL Properties

Method	Number of Unique Sequences	False Negatives Resolved
EchoFuzz	9	0/2
EchoFuzz+LTL	**13**	**2/2**

VFCS and its Property. There exists a gap between our generated candidate VFCS and the ideal VFCS properties. Our chain-guided procedure represents one specific implementation approach to approximate these ideal properties as closely as possible. Current methods focus on capturing vulnerability-triggering logical information, while determining ground truth for other properties remains extremely challenging. For example, verifying minimality requires knowing the exact minimal sequence, which is impractical without exhaustive exploration, though compromise approaches exist such as testing all subsequences of bug-triggering sequences to identify practically minimal candidates for experimental purposes.

7 Related Work

Smart Contract Fuzzing. Early tools like ContractFuzzer [24] generate random inputs and monitor contract execution for signs of vulnerabilities. Although effective at a basic level, random fuzzing lacks guidance, often exploring irrelevant or unreachable paths and resulting in inefficient coverage. Tools such as sFuzz [33] and Harvey [51] improve upon random fuzzing by incorporating coverage feedback to guide the generation of inputs. By focusing on unexplored paths, these fuzzers enhance efficiency. However, they often struggle to penetrate deeper contract states and may plateau after initial exploration phases. To overcome static and dynamic approach limitations, hybrid fuzzers like ConFuzzius [45] and ILF [18] integrate symbolic execution with fuzzing to expand path exploration and generate more relevant testcases. For example, MuFuzz [37] uses sequence-aware and mask-guided seed mutation to reach deeply nested states overlooked by others EchoFuzz focuses on the target of ideal VFCS, leveraging LLMs for contract-specific logical extraction to boost precision and cut exploration costs, while iterating on real-time fuzzing data to purposefully guide untapped path exploration.

LLM for Fuzzing. The integration of LLMs into fuzzing has demonstrated significant potential across domains. For example, LLM4Fuzz [41] leverages LLMs to prioritize high-risk code regions in smart contracts, achieving notable efficiency gains in vulnerability detection through guided exploration. Similarly, TitanFuzz [11] employs generative and infilling LLMs to synthesize input programs for deep learning libraries, surpassing traditional fuzzers in code coverage by up to 50%. These works highlight the capability of LLMs to reduce exploration overhead and enhance input diversity. EchoFuzz differs from existing approaches in two key aspects:

Analysis Emphasis. LLM4Fuzz prioritizes risky code via static patterns, while EchoFuzz focuses on function call sequences by understanding function interactions and state effects. For instance, while LLM4Fuzz relies on static analysis and LLM-generated metrics, it struggles to capture dynamic state dependencies (e.g., how `invest` and `refund` offset each other's effects in a `fundraiser` contract, in Section 3). EchoFuzz generates VFCS using a chain-guided procedure that analyzes state-function relationships to systematically construct vulnerability-triggering sequences.

Bottleneck Handling. While LLM4Fuzz enhances coverage and vulnerability detection via the initial input generated by LLM, it faces challenges to further improve coverage and detect additional vulnerabilities after the fuzzing process reaches a bottleneck. EchoFuzz introduces an LLM-guided refinement loop that iteratively optimizes sequence based on execution feedback. This enables dynamic adaptation to complex contract behaviors, crucial for overcoming exploration bottlenecks in stateful environments.

8 Conclusion

In this work, we present EchoFuzz, a novel LLM-guided fuzzing framework designed specifically for smart contracts. By leveraging the logical reasoning capabilities of LLMs, EchoFuzz effectively identifies candidate VFCS – a critical component for efficient smart

contract fuzzing. The framework first employs a chain-guided procedure that mimics the reasoning of expert auditors. This approach combines static analysis with few-shot prompting to enable the LLM to interpret contract behavior and propose high-quality initial VFCS candidates. Additionally, it employs a feedback-driven iterative process where the LLM analyzes real-time fuzzing results to adaptively refine and generate new VFCS candidates. This dynamic guidance directs the fuzzer to unexplored, vulnerable branches. Experiments demonstrate that EchoFuzz significantly outperforms existing approaches, achieving notable improvements in both coverage and vulnerability detection.

Acknowledgments

We would like to thank Jie Liang, Yu Jiang, Luke Ong, and Andrzej Murawski for their valuable feedback and support. We are also grateful to the anonymous reviewers for their constructive comments. This work was supported in part by the National Key Research and Development Project under Grant No. 2024YFF1401300, the Fundamental Research Funds for the Central Universities KG16358401, NSFC Program No. 62302256, No. 62502254 and NRF RSS Scheme NRF-RSS2022-009.

References

[1] Richard Amankwah, Patrick Kwaku Kudjo, and Samuel Yeboah Antwi. 2017. Evaluation of software vulnerability detection methods and tools: a review. *International Journal of Computer Applications* 169, 8 (2017), 22–27.

[2] Andreas M Antonopoulos and Gavin Wood. 2018. *Mastering ethereum: building smart contracts and dapps*. O'reilly Media.

[3] Parma Bains, Arif Ismail, Fabiana Melo, and Nobuyasu Sugimoto. 2022. *Regulating the crypto ecosystem: The case of unbacked crypto assets*. International Monetary Fund.

[4] Richard Bonett, Kaushal Kafle, Kevin Moran, Adwait Nadkarni, and Denys Poshyvanyk. 2018. Discovering flaws in {Security-Focused} static analysis tools for android using systematic mutation. In *27th USENIX Security Symposium (USENIX Security 18)*. 1263–1280.

[5] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2021. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering* 48, 9 (2021), 3280–3296.

[6] Yuanliang Chen, Fuchen Ma, Yuanhang Zhou, Yu Jiang, Ting Chen, and Jiaguang Sun. 2023. Tyr: Finding consensus failure bugs in blockchain system with behaviour divergent model. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2517–2532.

[7] Usman W Chohan. 2021. Non-fungible tokens: Blockchains, scarcity, and value. In *Non-Fungible Tokens*. Routledge, 1–11.

[8] Jaeseung Choi, Doyeon Kim, Soomin Kim, Gustavo Grieco, Alex Groce, and Sang Kil Cha. 2021. Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 227–239.

[9] Tatiana Cutts. 2019. Smart contracts and consumers. *W. Va. L. Rev.* 122 (2019), 389.

[10] Weiqi Dai, Qinyuan Wang, Zeli Wang, Xiaobin Lin, Deqing Zou, and Hai Jin. 2021. Trustzone-based secure lightweight wallet for hyperledger fabric. *J. Parallel and Distrib. Comput.* 149 (2021), 66–75.

[11] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.

[12] Etherscan Team. 2024. *Etherscan: Block Explorer and Analytics for Ethereum*. https://etherscan.io/

[13] João F. Ferreira, Pedro Cruz, Thomas Durieux, and Rui Abreu. 2021. SmartBugs: a framework to analyze solidity smart contracts. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (Virtual Event, Australia) *(ASE '20)*. Association for Computing Machinery, New York, NY, USA, 1349–1352. https://doi.org/10.1145/3324884.3415298

[14] Gustavo Grieco, Will Song, Artur Cygan, Josselin Feist, and Alex Groce. 2020. Echidna: effective, usable, and fast fuzzing for smart contracts. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 557–560.

[15] Oliver Hart and John Moore. 1988. Incomplete contracts and renegotiation. *Econometrica: Journal of the Econometric Society* (1988), 755–785.

[16] Oliver D Hart and Bengt Holmstrm. 1986. The theory of contracts. (1986).

[17] Haya R Hasan and Khaled Salah. 2018. Proof of delivery of digital assets using blockchain and smart contracts. *IEEE Access* 6 (2018), 65439–65448.

[18] Jingxuan He, Mislav Balunović, Nodar Ambroladze, Petar Tsankov, and Martin Vechev. 2019. Learning to fuzz from symbolic execution with application to smart contracts. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 531–548.

[19] Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. 2022. Large language models can self-improve. *arXiv preprint arXiv:2210.11610* (2022).

[20] Yongfeng Huang, Yiyang Bian, Renpu Li, J Leon Zhao, and Peizhong Shi. 2019. Smart contract security: A software lifecycle perspective. *IEEE Access* 7 (2019), 150184–150202.

[21] Florian Idelberger, Guido Governatori, Régis Riveret, and Giovanni Sartor. 2016. Evaluation of logic-based smart contracts for blockchain systems. In *Rule Technologies. Research, Tools, and Applications: 10th International Symposium, RuleML 2016, Stony Brook, NY, USA, July 6-9, 2016. Proceedings 10*. Springer, 167–183.

[22] Viktor Jacynycz, Adrian Calvo, Samer Hassan, and Antonio A Sánchez-Ruiz. 2016. Betfunding: A distributed bounty-based crowdfunding platform over ethereum. In *Distributed computing and artificial intelligence, 13th international conference*. Springer, 403–411.

[23] Songyan Ji, Jian Dong, Jin Wu, and Lishi Lu. 2023. A Guided Mutation Strategy for Smart Contract Fuzzing. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 282–292. https://doi.org/10.1109/ICSME58846.2023.00036

[24] Bo Jiang, Ye Liu, and Wing Kwong Chan. 2018. Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 259–269.

[25] Ahmed Kosba, Andrew Miller, Elaine Shi, Zikai Wen, and Charalampos Papamanthou. 2016. Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In *2016 IEEE symposium on security and privacy (SP)*. IEEE, 839–858.

[26] Bixin Li, Zhenyu Pan, and Tianyuan Hu. 2024. EvoFuzzer: An Evolutionary Fuzzer for Detecting Reentrancy Vulnerability in Smart Contracts. *IEEE Transactions on Network Science and Engineering* (2024), 1–14. https://doi.org/10.1109/TNSE.2024.3447025

[27] Jun Li, Bodong Zhao, and Chao Zhang. 2018. Fuzzing: a survey. *Cybersecurity* 1 (2018), 1–13.

[28] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. Vulpecker: an automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd annual conference on computer security applications*. 201–213.

[29] Zhenguang Liu, Peng Qian, Jiaxu Yang, Lingfeng Liu, Xiaojun Xu, Qinming He, and Xiaosong Zhang. 2023. Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting. *IEEE Transactions on Information Forensics and Security* 18 (2023), 1237–1251.

[30] Fuchen Ma, Meng Ren, Ying Fu, Mingzhe Wang, Huizhong Li, Houbing Song, and Yu Jiang. 2021. Security reinforcement for Ethereum virtual machine. *Information Processing & Management* 58, 4 (2021), 102565.

[31] Fuchen Ma, Meng Ren, Lerong Ouyang, Yuanliang Chen, Juan Zhu, Ting Chen, Yingli Zheng, Xiao Dai, Yu Jiang, and Jiaguang Sun. 2023. Pied-piper: Revealing the backdoor threats in ethereum erc token contracts. *ACM Transactions on Software Engineering and Methodology* 32, 3 (2023), 1–24.

[32] Robert C Newman. 2009. *Computer Security: Protecting Digital Resources: Protecting Digital Resources*. Jones & Bartlett Publishers.

[33] Tai D Nguyen, Long H Pham, Jun Sun, Yun Lin, and Quang Tran Minh. 2020. sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 778–788.

[34] Hannu Nurmi. 2012. *Comparing voting systems*. Vol. 3. Springer Science & Business Media.

[35] Manjula K Pawar, Prakashgoud Patil, Manisha Sharma, and Megha Chalageri. 2021. Secure and scalable decentralized supply chain management using Ethereum and IPFS platform. In *2021 International Conference on Intelligent Technologies (CONIT)*. IEEE, 1–5.

[36] Federico Pigni, Marcin Bartosiak, Gabriele Piccoli, and Blake Ives. 2018. Targeting Target with a 100 million dollar data breach. *Journal of Information Technology Teaching Cases* 8, 1 (2018), 9–23.

[37] Peng Qian, Hanjie Wu, Zeren Du, Turan Vural, Dazhong Rong, Zheng Cao, Lun Zhang, Yanbin Wang, Jianhai Chen, and Qinming He. 2024. MuFuzz: Sequence-Aware Mutation and Seed Mask Guidance for Blockchain Smart Contract Fuzzing. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1972–1985.

[38] Meng Ren, Fuchen Ma, Zijing Yin, Ying Fu, Huizhong Li, Wanli Chang, and Yu Jiang. 2021. Making smart contract development more secure and easier. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering*

Conference and Symposium on the Foundations of Software Engineering. 1360–1370.

[39] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 757–762.

[40] Andrea Sestino, Gianluigi Guido, Alessandro M Peluso, et al. 2022. Non-fungible tokens (NFTs). *Examining the Impact on Consumers and Marketing Strategies* (2022).

[41] Chaofan Shou, Jing Liu, Doudou Lu, and Koushik Sen. 2024. Llm4fuzz: Guided fuzzing of smart contracts with large language models. *arXiv preprint arXiv:2401.11108* (2024).

[42] Khaled Shuaib, Juhar Ahmed Abdella, Farag Sallabi, and Mohammed Abdel-Hafez. 2018. Using blockchains to secure distributed energy exchange. In *2018 5th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, 622–627.

[43] Settaluri Lakshmi Sravanthi, Meet Doshi, Pavan Kalyan Tankala, Rudra Murthy, and Pushpak Bhattacharyya. [n. d.]. Do LLMs understand Pragmatics? An Extensive Benchmark for Evaluating Pragmatic Understanding of LLMs. ([n. d.]).

[44] Jianzhong Su, Hong-Ning Dai, Lingjun Zhao, Zibin Zheng, and Xiapu Luo. 2022. Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.

[45] Christof Ferreira Torres, Antonio Ken Iannillo, Arthur Gervais, and Radu State. 2021. Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 103–119.

[46] Foteini Valeonti, Antonis Bikakis, Melissa Terras, Chris Speed, Andrew Hudson-Smith, and Konstantinos Chalkias. 2021. Crypto collectibles, museum funding and OpenGLAM: challenges, opportunities and the potential of Non-Fungible Tokens (NFTs). *Applied Sciences* 11, 21 (2021), 9931.

[47] Haijun Wang, Yi Li, Shang-Wei Lin, Lei Ma, and Yang Liu. 2019. Vultron: catching vulnerable smart contracts once and for all. In *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 1–4.

[48] Sam Werner, Daniel Perez, Lewis Gudgeon, Ariah Klages-Mundt, Dominik Harz, and William Knottenbelt. 2022. Sok: Decentralized finance (defi). In *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*. 30–46.

[49] Jan Wloka, Manu Sridharan, and Frank Tip. 2009. Refactoring for reentrancy. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on the Foundations of Software Engineering*. 173–182.

[50] Gavin Wood et al. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 151, 2014 (2014), 1–32.

[51] Valentin Wüstholz and Maria Christakis. 2020. Harvey: A greybox fuzzer for smart contracts. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1398–1409.

[52] Min Yang, Qiang Qu, Wenting Tu, Ying Shen, Zhou Zhao, and Xiaojun Chen. 2019. Exploring human-like reading strategy for abstractive text summarization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 7362–7369.

[53] Dirk A Zetzsche, Douglas W Arner, and Ross P Buckley. 2020. Decentralized finance. *Journal of Financial Regulation* 6, 2 (2020), 172–203.

[54] Wuqi Zhang, Zhuo Zhang, Qingkai Shi, Lu Liu, Lili Wei, Yepang Liu, Xiangyu Zhang, and Shing-Chi Cheung. 2024. Nyx: Detecting Exploitable Front-Running Vulnerabilities in Smart Contracts. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 146–146.

[55] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).

[56] Zibin Zheng, Shaoan Xie, Hong-Ning Dai, Weili Chen, Xiangping Chen, Jian Weng, and Muhammad Imran. 2020. An overview on smart contracts: Challenges, advances and platforms. *Future Generation Computer Systems* 105 (2020), 475–491.